Ingeniería de Software Asistida por IA

Tema 4: Scaffolding, Patrones y Contratos Duros

Guía de Estudio Detallada para Estudiantes

¿Qué es el Scaffolding Arquitectónico?

El *Scaffolding* (Andamiaje) es la práctica innegociable de crear físicamente la estructura de carpetas y los archivos vacíos de un proyecto **antes** de ordenarle a la IA que escriba código.

Si tú le pides a una IA que "cree un sistema de carrito de compras" sobre un repositorio vacío, la máquina generará un único archivo monstruoso que contiene la interfaz visual, las validaciones de negocio y las llamadas directas a la base de datos. Esto viola todos los principios de la ingeniería. Al crear el Scaffolding primero (ej. `mkdir components hooks services`), estás encerrando a la IA en una "Jaula Arquitectónica". Cuando le dices *"Rellena el archivo `/services/cart.ts`"*, le imposibilitas físicamente mezclar esa lógica con el diseño visual de `/components`.

Diseccionando el Frontend Moderno

Un Scaffolding profesional aísla la interfaz de la lógica de red. Si cambias de base de datos mañana, tus componentes visuales ni siquiera deberían enterarse.

- **`/components/ui`:** Componentes "tontos" (Dumb components). Botones, modales, tarjetas. No tienen lógica de base de datos. Solo reciben datos (`props`) y los dibujan. Son altamente reutilizables.
- **`/hooks`:** Lógica de estado complejo (ej. `useAuth`). Mantiene el estado en la memoria del navegador.
- **`/services`: La Zona Crítica.** Es la única capa autorizada para hacer peticiones (`fetch`) a la API o interactuar con servicios externos.
- **`/types` (o `/interfaces`):** El corazón de tu seguridad tipada. Define la forma exacta de tus objetos.

Contratos Duros (Interfaces como Ley)

Las IAs tienden a "inventar" propiedades en los objetos por conveniencia. Para evitar esto, el primer paso en cualquier desarrollo debe ser definir los **Contratos** (ej. Interfaces en TypeScript o validadores en Zod).

Si creas una interfaz `User { id: string; email: string; }`, este contrato se convierte en ley. Al pasarle este contrato a la IA y ordenarle que lo respete, usas a tu propio entorno de desarrollo (tu compilador) como auditor. Si la IA intenta usar `User.password` en su lógica por error, tu compilador lanzará un error crítico inmediato. Los contratos duros eliminan la ambigüedad y someten la alucinación de la máquina a las reglas de la matemática estricta de tipos.

El Patrón Singleton (Defensa de Infraestructura)

El Patrón Singleton asegura que una clase o conexión tenga una **única instancia compartida** en todo el ciclo de vida de la aplicación. En el desarrollo Serverless (como Deno o Supabase Edge Functions), omitir este patrón es letal.

Si la IA programa la conexión a tu base de datos libremente sin usar Singleton, instanciará un nuevo cliente de base de datos por cada usuario que haga clic en tu página web. En 5 minutos, saturarás el *Connection Pool* de tu base de datos y tu sistema colapsará por completo.

> ***Prompt Obligatorio:*** *"Para conectarte a la base de datos, implementa el Patrón Singleton en un archivo `/_shared/db.ts`. Garantiza que si la conexión ya existe, se reutilice, en lugar de instanciarse nuevamente."*

El Patrón Repository

La regla fundamental del software robusto es: **El Controlador o la Interfaz jamás hablan con SQL.**

El Patrón Repository obliga a centralizar todas las operaciones de la base de datos en un solo archivo. Por ejemplo, `UserRepository.ts` contendrá funciones abstractas como `getUserById()`. Si en el futuro la empresa decide migrar de PostgreSQL a MongoDB, tú, como Arquitecto, solo tendrás que reescribir ese único archivo (`UserRepository.ts`). El resto de tu aplicación (las vistas, los hooks, los controladores) seguirá funcionando intacto porque solo saben llamar a `getUserById()`. La IA odia crear abstracciones porque gasta tokens; tú debes obligarla a usarlas.